



[BIT703 Web Technologies](#) / [Contents](#) / Topic 7: Client-side web application development and programming – JavaScript / Basics of JavaScript

JavaScript variables 20m

Variables in JavaScript are containers for storing data. For example, a JavaScript variable can be used to store a user's phone number or full name.

There are three ways to define a variable in JavaScript:

- var
- const
- let

Variable	Description	Examples
var	► 'var' is used to declare variables in legacy code (code from 1995 to 2015). ► 'var' is a global variable, meaning it is accessible throughout the whole file or function in which it is declared. ► A variable can be redeclared using 'var'.	<pre>var x = 5 var y = 7 var z = x + y</pre>
const	► 'const' is a local variable, meaning its scope is defined by the block in which it appears. ► A variable cannot be redeclared using 'const'. ► 'const' is used if you want to declare a variable that will not change.	<pre>const price1 = 5 const price2 = 7</pre>

[Skip to main content](#)

let	► 'let' is a local variable, meaning its scope is defined by the block in which it appears. ► A variable cannot be redeclared using 'let'. ► 'let' is used if the value of the variable is likely to change.	<pre>let total = price1 + price2</pre>
-----	--	--

A variable can contain different types of JavaScript data, such as arrays, objects, numbers and true or false Boolean. We will learn about these next.

Note: The JavaScript language is case-sensitive, so be extra careful when naming variables and functions and using language statements.

For example, 'var SomeValue' is not the same as 'var some_value'.

JavaScript strings

Strings in JavaScript are used to store and manipulate text. A JavaScript string is text that is enclosed in quotes. Strings can be enclosed in either matching single ('single quote') or double quotes ('double quote').

For example, imagine you have a JavaScript variable for first name:

```
var firstName = 'Jennifer'
```

You can see that this has created a string that will be displayed as Jennifer.

The fact that a string can have single or double quotes comes in handy when your string includes a quote or when you need to use a JavaScript string within some HTML attribute that also uses a pair of quotes to enclose the values.

For example, because the string below has double quotes within a string of double quotes, JavaScript will have trouble understanding this string:

[Skip to main content](#)

```
let text = "Tony said "should we go to the supermarket before the movies?" to his friends as they walked to the movies"
```

In this case, you'd use single quotes to enclose your string, so you aren't matching the type of quotes used.

As long as you don't match the quotes used to enclose the string, you can use quotes inside the string.

Alternatively, another option in this situation is to use backslash escape characters.

- The code `\'` can be used to create a single quote within a string.
- The code `\"` can be used to create a double quote within a string.

You can link multiple strings and data types (such as strings with numbers) with the `+` character.

For example, take the example above for `var firstName`. We can concatenate another string to this by using the `+` character:

```
var firstName = 'Jennifer'

var lastName = 'Jones'

var finalString = firstName + lastName
```



[Skip to main content](#)

Dig deeper

To learn more about JavaScript strings, refer to these resources:

- String basics – Beau teaches JavaScript (freeCodeCamp.org, 2017) (3:31) this video demonstrates how to perform basic JavaScript functions with strings.
- JavaScript Strings (W3Schools.com, n.d.) demonstrates examples of JavaScript strings.



JavaScript numbers and arrays

Numbers

Unlike many other languages that have different numerical types, such as integer, float and double, JavaScript only has one numerical type, 'number', which is extremely versatile.

Numbers can have a floating decimal point or fraction and they can also be scientific with an 'e' for the exponent; for example:

```
var myExponent1 = 123e-5; will produce the number 0.00123
```

While `var myExponent2 = 123e5;` will produce the number 12300000.

As you can see, the 'e' for exponent moves the decimal point an X number of spaces. The X number is determined by the number presented after the 'e' in the JavaScript expression.

Booleans

Sometimes in programming, you need a data type that can only have one of two values. These are situations such as:

[Skip to main content](#)

- **yes or no**
- **on or off**
- **true or false.**

In these situations, you can use a Boolean data type because they only take values that are true or false. Booleans are useful in controlling program flow using conditional statements like if...else..., switch, while loop, etc.

A Boolean is simply true or false, but you must remember that literals are case-sensitive and must be written in lowercase:

```
var myBool = true; myBool = false;
```



Dig deeper

To learn more about Booleans, refer to these resources:

- JavaScript Boolean (GeeksforGeeks, 2024) provides examples and explanations of different JavaScript Booleans
- JavaScript Booleans Tutorial (Tutorials Teacher, n.d.) provides a tutorial for how to write and work with Booleans.



Arrays

Arrays in JavaScript are special variables that can hold more than one value. You use an array to store multiple items under a single variable. JavaScript arrays can contain mixed data types because they are collections of objects. The syntax for an array is:
[Skip to main content](#)

```
var array_name = [item1, item2, ... ];
```

JavaScript arrays are useful because not only do they keep your code tidy by grouping your items under a single variable, but you can also access a specific value in the array by referring to its index number; this is useful when you have multiple items, like 300 items.

Imagine you have a list of items; for example, a list of fruit names. You could store each fruit as a single variable:

```
var fruit1 = apple
```

```
var fruit2 = banana
```

```
var fruit3 = pear
```

However, it would be better to create an array for these fruit items; your array would look like this:

```
var fruitArray = ["apple", "banana", "pear"];
```

Accessing array elements

You can access the values in an array by referring to their index number. Arrays have zero-based indexes, which means the first item has an index of 0, not 1.

The first item in an array starts with the index number 0, the second item has an index of 1, the third item has an index of 2, and so on.

The syntax to access an array element is:

```
array_name[index]
```

For example, imagine we want to access the item 'banana' in the array `var fruitArray = ["apple", "banana", "pear"]`. To do this, we'd write:

```
fruitArray[1]
```

[Skip to main content](#)

This would produce the output: banana.



Dig deeper

To learn more about arrays and see examples of how to create arrays and access their items, refer to these resources:

- JavaScript Arrays (Programiz, n.d.)
- JavaScript Arrays (JavaScript Tutorial, n.d.).



Thoughts on this page?

